

WHAT THIS SESSION IS

Run on 5 September 2026. The previous class stopped at an uncomfortable place: the model asked for a tool, and nothing happened. This session closes that gap. First the tool calling loop is written by hand, every step of it, so you can see exactly what an agent does. Then the same loop is replaced by `create_agent`, and from there the session builds up quickly: four tools with the agent choosing between them, streaming so you can watch it work, memory so a conversation survives across calls, and finally a small support agent that puts all of it together with validated input and a structured reply. Six files, and by the end the word "agent" has stopped being a buzzword.

CONTENTS

1. Where this session picks up
2. Reading the tool call by hand
3. The tool calling loop, written by hand
4. What the message trail tells you
5. `create_agent`: the same loop in three lines
6. Four tools, and letting the agent choose
7. Watching the agent work with streaming
8. Memory: checkpointers and thread ids
9. Where LangGraph fits into all this
10. The mini project: a support agent
11. Structured output, at the agent level
12. Clarifications from the Q&A
13. Key takeaways

1 Where this session picks up

Last class ended with `bind_tools` working and an answer that never arrived. Asked how many words are in a sentence, the model replied with an empty `content` and a filled `tool_calls` list naming `word_count` and the exact argument to pass. The tool was requested. It was never run.

WHERE WE STOPPED

```
content      ''                                ← empty
tool_calls [{"name": "word_count",
              'args': {'text': 'LangChain makes agents simple'},
              'id':   'call_abc123'}]          ← a request, not a result
```

the model did its job. it decided, and it asked.
nobody executed anything, so nobody sent a result back,
so the model never got to write the final sentence.

Everything in this session follows from filling in that gap.

ONE HOUSEKEEPING FIX FROM LAST WEEK

The `current_time` tool failed last class with "No time zone found with key", because Python on Windows ships without a time zone database. It is now a declared dependency of the project, so it is installed once and forgotten:

```
uv add tzdata
```

2 Reading the tool call by hand

Before writing a loop, the class did the simplest possible version: take the reply, check whether a tool was requested, and pull the details out.

```
reply = model_with_tools.invoke(
    "How many words are in the sentence: LangChain makes agents simple?"
)

if reply.tool_calls:

    tool_call = reply.tool_calls[0]      # the first tool it asked for

    print("requested tool:", tool_call["name"])
    print("arguments      :", tool_call["args"])
```

Two things worth naming here, because both were asked in the room.

- **Why index zero.** Because at this stage we are only looking at the first request. A model can ask for several tools in one turn, which is exactly what the next section handles properly.
- **Who fills in those arguments.** The model. It reads the tool's description and the user's sentence, and produces the argument dictionary. That is the reason validation matters, and it is why the previous class put Pydantic in front of the function body.

3 The tool calling loop, written by hand

This is the centre of the session. Written once by hand, everything after it stops being magic.

```

+-----+
messages ---> |           THE MODEL           |
^             | reads the whole conversation |
|             | and decides what to do next  |
|             +-----+
|             |
|             +-----+
|             |                               |
|             |                               |
|             | asked for tools?             | asked for nothing?
|             |                               |
|             | v                             | v
|             +-----+                     +-----+
|             | YOUR CODE runs |             | this is the |
|             | each tool it   |             | FINAL ANSWER |
|             | named          |             | break the loop |
|             +-----+                     +-----+
|             |
|             | v
|             | each result becomes a
|             | ToolMessage, appended
|             | to the conversation
|             |
+-----+

```

send, act, observe, send again. that cycle is the ReAct pattern,
from REasoning and ACTing. every agent framework is a version of it.

tool_calling_loop.py

```
import sys
from datetime import datetime
from zoneinfo import ZoneInfo

from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, SystemMessage

load_dotenv()

# Just makes the console support UTF-8 characters.
sys.stdout.reconfigure(encoding="utf-8")

@tool
def current_time(city: str) → str:
    """Get the current time in a city."""

    zones = {
        "mumbai": "Asia/Kolkata",
        "london": "Europe/London",
        "new york": "America/New_York"
    }

    zone = zones.get(city.lower())

    if zone is None:
        return f"I do not know the timezone for {city}."

    return datetime.now(ZoneInfo(zone)).strftime("%d %B %Y, %I:%M %p")

@tool
def multiply(a: int, b: int) → int:
    """Multiply two numbers and return the exact result."""
    return a * b
```

```
# put our tools into a list
tools = [current_time, multiply]

# we need the name the LLM gives us to find the actual Python tool
tools_by_name = {t.name: t for t in tools}

model = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0
).bind_tools(tools)
```

Note the last statement. In the previous class the model was built first and `bind_tools` applied on a second line. Here it is chained into one expression. Same result, fewer lines, and there is no reason to split it when nothing else uses the unbound model.

WHY THE DICTIONARY EXISTS

`tools_by_name` is the piece people skip and then get stuck on. The model does not hand you a function. It hands you a **string**, the tool's name. You need a lookup from that string back to the real Python object so you can run it. With one tool you can cheat and call it directly, which is what last class did. With two or twenty you need the map.

The conversation, then the loop

```
# create the conversation
messages = [
    SystemMessage(
        content="You are a helpful assistant. Use the tools when they fit."
    ),
    HumanMessage(
        content="What time is it in Mumbai, and what is 98765 times 43210?"
    ),
]

step = 1

while True:

    response = model.invoke(messages)

    # keep the model's own reply in the conversation, so the next
    # turn knows what it already said
    messages.append(response)

    if not response.tool_calls:
        print("\nFinal answer:")
        print(response.content)
        break

    print(f"Step {step}: the model asked for {len(response.tool_calls)} tool call(s)")

    for call in response.tool_calls:

        # call contains: tool name, arguments, tool call id
        tool_to_run = tools_by_name[call["name"]]

        # passing the whole call back gives a ToolMessage that is already
        # tied to this request by its id
        tool_message = tool_to_run.invoke(call)

        print(f"    {call['name']}({call['args']}) → {tool_message.content}")

        messages.append(tool_message)
```

```
step += 1
```

```
print("\nMessages in the conversation:", [type(m).__name__ for m in messages])
```

The five moves, in order

1. **Send the whole conversation** to the model, not just the latest line.
2. **Append the model's reply** to the conversation. Skip this and the next turn has no idea what the model already asked for.
3. **If no tools were requested**, this is the final answer. Print it and break.
4. **Otherwise run every tool it named**, not just the first. The question in this file deliberately needs two.
5. **Append each result** and go round again.

WHY `TOOL.INVOKE(CALL)` AND NOT `TOOL.INVOKE(CALL["ARGS"])`

Both run the function. The difference is what comes back. Pass only the arguments and you get the raw return value, a plain string or number, and then you have to wrap it into a `ToolMessage` and attach the correct `tool_call_id` yourself.

Pass the **whole call** and LangChain builds the `ToolMessage` for you, already carrying the id that ties this result to that request. The model needs that id to know which answer belongs to which question when it asked for several at once. One character of extra typing saves a class of confusing bugs.

4 What the message trail tells you

The question was *what time is it in Mumbai, and what is 98765 times 43210*? Run live, the output was:

Step 1: the model asked for 2 tool call(s)

```
current_time({'city': 'Mumbai'}) → 05 September 2026, 08:48 PM
```

```
multiply({'a': 98765, 'b': 43210}) → 4267595650
```

Final answer:

The current time in Mumbai is 8:48 PM on 5 September 2026, and
98765 times 43210 is 4,267,595,650.

Messages in the conversation:

```
['SystemMessage', 'HumanMessage', 'AIMessage', 'ToolMessage',  
 'ToolMessage', 'AIMessage']
```

That last line is the whole story of the run, and it is worth reading slowly.

Message	What it is
<code>SystemMessage</code>	The tone you set. Written by you.
<code>HumanMessage</code>	The question. Written by the user.
<code>AIMessage</code>	The model's first reply. Empty content, two tool calls.
<code>ToolMessage</code>	The result of <code>current_time</code> . Produced by your code.
<code>ToolMessage</code>	The result of <code>multiply</code> . Produced by your code.
<code>AIMessage</code>	The final answer, written after seeing both results.

WHERE THE ANSWER ACTUALLY COMES FROM

Asked directly in class, and it is a good question. The number came from the tool. The sentence came from the model. Neither could have produced that reply alone.

The tool cannot write English. The model cannot multiply reliably. What you built is a system where the exact part is exact and the readable part is readable, and the loop is what joins them.

TWO LOOPS RUN HERE, NOT ONE

The `while True` is the outer loop: one pass per round trip to the model. The `for call in response.tool_calls` is the inner loop: one pass per tool the model asked for in that round.

Ask for two tools in one turn and the inner loop runs twice while the outer loop runs once. Ask a question where the second tool depends on the first result, and the outer loop runs twice. Both were asked about in the room and it is easy to conflate them.

5 create_agent: the same loop in three lines

Having written it, you get to delete it.

BY HAND

```
tools_by_name = {...}
model = ChatOpenAI(...).bind_tools()
messages = [...]

step = 1
while True:
    response = model.invoke(messages)
    messages.append(response)
    if not response.tool_calls:
        print(response.content)
        break
    for call in response.tool_calls:
        t = tools_by_name[call["name"]]
        msg = t.invoke(call)
        messages.append(msg)
    step += 1
```

about fifteen lines

WITH create_agent

```
agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[current_time, multiply],
    system_prompt="...",
)

result = agent.invoke({
    "messages": [
        {"role": "user",
         "content": question}
    ]
})

print(result["messages"][-1].content)
```

three lines of real work

SAME BEHAVIOUR. the loop did not disappear, it moved.

agent_too_loop.py

```
import sys
from datetime import datetime
from zoneinfo import ZoneInfo

from dotenv import load_dotenv

# create_agent is going to manage the tool-calling loop for us.
from langchain.agents import create_agent
from langchain_core.tools import tool

load_dotenv()
sys.stdout.reconfigure(encoding="utf-8")

@tool
def current_time(city: str) → str:
    """Get the current time in a city."""
    zones = {
        "mumbai": "Asia/Kolkata",
        "london": "Europe/London",
        "new york": "America/New_York"
    }
    zone = zones.get(city.lower())
    if zone is None:
        return f"I do not know the timezone for {city}."
    return datetime.now(ZoneInfo(zone)).strftime("%d %B %Y, %I:%M %p")

@tool
def multiply(a: int, b: int) → int:
    """Multiply two numbers and return the exact result."""
    return a * b

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[current_time, multiply],
    system_prompt=(
        "You are a helpful assistant. "
```

```

        "Use the tools when they fit."
    ),
)

result = agent.invoke({
    "messages": [
        {
            "role": "user",
            "content": "What time is it in Mumbai, and what is 98765 times 43210?"
        }
    ]
})

print("Final Answer")
print(result["messages"][-1].content)

```

The shape of the thing

- **Input** is a dictionary with a `messages` key. Each message is `{"role": ..., "content": ...}`, the same shape you saw in the very first raw REST call in session 2.
- **Output** is also a dictionary with `messages`, and it contains the *whole conversation*, not just the answer.
- `result["messages"][-1].content` is therefore the final answer: the last message in the trail.
- **The model can be a string** here, `"openai:gpt-4o-mini"`, in the `provider:model` form. You can also pass a model object, which becomes useful in the next class.

Printing the trail

Everything the hand-written loop printed is still available, because the whole conversation comes back:

```

print("What happened along the way:")

for message in result["messages"]:

    kind = type(message).__name__

    if getattr(message, "tool_calls", None):
        print(f" {kind}: asked for {[c['name'] for c in message.tool_calls]}")
    else:
        print(f" {kind}: {str(message.content)[:70]}")

```

WHICH ONE SHOULD YOU USE

The answer given was refreshingly practical. If you want complete control over every step, write the loop. If you want the agent to handle it, use `create_agent`. It is a choice, not a progression, and the reason for writing the loop first was never that you would ship it.

In practice you will use `create_agent`, and the next session shows how to reach back into the loop without giving up the shortcut.

Reference: docs.langchain.com/oss/python/langchain/agents

6 Four tools, and letting the agent choose

Two tools proves the mechanism. Four starts to look like an application.

There is no database yet, so a couple of Python dictionaries stand in for one. That is deliberate: FastAPI and real databases come later in the course, and mixing them in now would bury the point.

moretool-agent.py

```
# our data → imagine it is our order database
ORDERS = {
    "ORD-1001": {"item": "Wireless mouse", "status": "shipped", "amount": 1499},
    "ORD-1002": {"item": "Mechanical keyboard", "status": "packed", "amount": 4999},
}

# imagine we have warehouse stock
STOCK = {"wireless mouse": 12, "mechanical keyboard": 0, "usb hub": 34}

@tool
def order_status(order_id: str) → str:
    """Get the status and amount of an order using its id, for example ORD-1001."""
    order = ORDERS.get(order_id.upper())
    if order is None:
        return f"No order found with id {order_id}."
    return f"{order['item']}, status {order['status']}, amount {order['amount']} rupees"

@tool
def check_stock(item: str) → str:
    """Check how many units of an item are left in the warehouse."""
    count = STOCK.get(item.lower())
    if count is None:
        return f"{item} is not in the catalogue."
    return f"{count} units of {item} in stock"

@tool
def apply_discount(amount: float, percent: float) → float:
    """Apply a discount percentage to an amount and return the new amount."""
    return round(amount - (amount * percent / 100), 2)

@tool
def delivery_days(pin_code: str) → str:
    """Estimate delivery time for an Indian pin code."""
    metro = {"400001", "110001", "560001"}
    return "2 days" if pin_code in metro else "5 days"
```

```

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[order_status, check_stock, apply_discount, delivery_days],
    system_prompt=(
        "You are a support assistant for an online store. "
        "Use the tools for anything about orders, stock, pricing or delivery. "
        "Never guess a number, look it up."
    ),
)

def ask(question):
    print("Q:", question)
    result = agent.invoke({"messages": [{"role": "user", "content": question}]})
    print("A:", result["messages"][-1].content)
    print("-" * 60)

```

Four questions, four different behaviours

Question	What happened	Why it matters
What is the status of order ORD-1001	One tool: <code>order_status</code>	The straightforward case
Do you have a mechanical keyboard in stock	One tool: <code>check_stock</code> , answered zero units	Picked the right tool out of four, from the descriptions
When will the mechanical keyboard be available again	No tool fits. It said it does not have that information	See the callout below
For ORD-1002, what would the price be after a 10 percent discount	Two tools in sequence: <code>order_status</code> , then <code>apply_discount</code>	Nobody wrote that plan

THE THIRD QUESTION IS THE INTERESTING ONE

It was asked because students in the chat wanted to know what happens when nothing fits. There is no restock tool, and the data does not contain a restock date.

It did not invent one. It said it does not have that information. That is the behaviour you want, and it did not happen by accident: the system prompt says *never guess a number, look it up*. Prompt rules are part of the design, not decoration.

THE FOURTH QUESTION IS THE AGENTIC ONE

Applying a discount needs an amount. The question did not contain one, it contained an order id. So the agent looked up the order, took the amount from that result, and fed it into the discount tool.

That is two round trips through the loop, with the second call's arguments coming from the first call's result. You did not write that sequence anywhere. The model worked it out from the two descriptions. This is the first point in the course where the word "agent" is doing real work.

7 Watching the agent work with streaming

`invoke` gives you the answer after everything is finished. While you are building, you want to see the loop turning.

```

@tool
def ticket_price(city: str) → int:
    """Get the flight ticket price in rupees from Mumbai to a city."""
    prices = {"delhi": 4500, "bengaluru": 3900, "kolkata": 5200}
    return prices.get(city.lower(), 6000)

@tool
def hotel_price(city: str, nights: int) → int:
    """Get the total hotel cost in rupees for a number of nights in a city."""
    per_night = {"delhi": 3000, "bengaluru": 3500, "kolkata": 2800}
    return per_night.get(city.lower(), 3200) * nights

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[ticket_price, hotel_price],
    system_prompt="You plan small trips and always look up real numbers with the tools.",
)

question = "I want to go to Bengaluru for 3 nights from Mumbai. What is my total cost?"

for chunk in agent.stream(
    {"messages": [{"role": "user", "content": question}]},
    stream_mode="updates"  # give me an update as each step happens
):
    for node, update in chunk.items():
        for message in update["messages"]:

            if getattr(message, "tool_calls", None):
                print(f"[{node}] wants: {[c['name'] for c in message.tool_calls]}")

            elif type(message).__name__ == "ToolMessage":
                print(f"[{node}] {message.name} returned: {message.content}")

            elif message.content:
                print(f"[{node}] says: {message.content}")

```

```
[model] wants: ['ticket_price', 'hotel_price']  
[tools] ticket_price returned: 3900  
[tools] hotel_price returned: 10500  
[model] says: Your trip to Bengaluru from Mumbai for 3 nights costs ...
```

one line per step. "model" and "tools" alternate,
which is exactly the loop from section 3.

THE TWO WORDS THAT CARRY THIS FILE

`stream_mode="updates"` is not the same streaming as session 4. There, `model.stream` gave you the answer text word by word. Here you get one chunk per *step of the agent*, keyed by which node ran.

`node` is that key. It will read `model` or `tools`, and seeing them alternate is seeing the loop turn.

Say it plainly to yourself: this is the debugging tool for the rest of the course. When an agent misbehaves you do not guess, you stream it and look at the steps.

8 Memory: checkpoints and thread ids

Every `invoke` so far started from nothing. Tell the agent your name, ask for it on the next call, and it has no idea. That is not a bug. The agent receives the messages you pass and nothing else.

A **checkpointer** changes that. It saves the conversation after every step, filed under a **thread id** that you choose.

agent-tool-memory.py

```
from langgraph.checkpoint.memory import InMemorySaver

@tool
def book_seat(name: str, seat: str) → str:
    """Book a seat in the class for a person."""
    return f"Seat {seat} booked for {name}."

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[book_seat],
    system_prompt="You help learners book a seat in the LangChain class.",
    checkpoint=InMemorySaver(), # tells the agent to store the conversation after each
    step
)

def ask(question, thread_id):
    result = agent.invoke(
        {"messages": [{"role": "user", "content": question}]},
        config={"configurable": {"thread_id": thread_id}},
    )
    print(f"[{thread_id}] Q: {question}")
    print(f"[{thread_id}] A: {result['messages'][-1].content}")
    print()

ask("My name is vinay kumar gurram", thread_id="vinay")
ask("Book me a seat A12.", thread_id="vinay")
ask("What is my name and what seat did i book", thread_id="vinay")
ask("What is my name and what seat did i book", thread_id="nimish")
```

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

THREAD "vinay"

turn 1 "my name is Vinay Kumar
Gurram"
→ nice to meet you

turn 2 "book me a seat A12"
→ seat A12 booked for
Vinay Kumar Gurram

turn 3 "what is my name and
what seat did I book"
→ your name is Vinay Kumar
Gurram and you booked A12

turn 3 never repeated the name or the seat.
the thread carried them. a different thread knows nothing,
which is how one running application serves many customers.

THREAD "nimish"

turn 1 "what is my name and
what seat did I book"
→ I do not have that
information

THE THREAD ID IS AN ID, NOT A NAME

It reads like a name in this demo only because the trainer used "vinay" for both, and he called that out on air after a student got confused. The thread id is just a key. In a real application it would be a session id, a customer id, or something generated per conversation. The agent never reads it as content.

INMEMORYSAVER IS NOT FOR PRODUCTION

Said explicitly, and it matches what the official documentation says. It keeps checkpoints in RAM, so everything is gone when the process exits, and it does not survive more than one running instance.

For a real application you swap it for a SQLite or Postgres saver. Nothing else in the file changes, because the checkpointer sits in the same slot. Storing conversations in a proper database is planned for a later session.

Reference: docs.langchain.com/oss/python/langgraph/persistence

9 Where LangGraph fits into all this

That import line caused a stir in the chat, and it deserves a proper answer:

```
from langgraph.checkpoint.memory import InMemorySaver
```

Several people asked why LangGraph appeared when the class is about LangChain, and whether memory requires LangGraph. The answer given was clear.

CREATE_AGENT IS BUILT ON LANGGRAPH

`create_agent` is not a hand-rolled while loop hidden in LangChain. It is built on LangGraph's runtime and state machinery. So the agent APIs you have been using this whole session are, underneath, LangGraph APIs. The checkpointer comes from there because persistence is a LangGraph concept.

Which means you have already been using a little LangGraph without noticing, and that is the intended way in.

The wider point, made as advice rather than syllabus: if you are building agentic systems in Python, LangChain alone is not enough. You will need LangGraph as soon as you want several agents, explicit workflows, conditional routing or human approval in the middle of a run. It was also described as far less intimidating than people expect, with two or three weekends being enough once LangChain is comfortable.

what you have used		what it sits on
<code>create_agent(...)</code>	--->	LangGraph runtime, state, nodes
<code>checkpointer=...</code>	--->	LangGraph persistence
<code>stream_mode="updates"</code>	--->	LangGraph step updates

coming later in the course:

- nodes and edges written out explicitly
- conditional routing
- human in the loop
- multi-agent workflows

10 The mini project: a support agent

The last file puts everything from the session into one small application: four tools, validated input, memory across turns, and a structured reply.

Validated tool input

```
from pydantic import BaseModel, Field

# pydantic model for tool input, so we do not blindly trust the argument
class RefundInput(BaseModel):

    order_id: str = Field(
        description="Order id, for example ORD-1001"
    )

    reason: str = Field(
        description="Why the customer wants a refund",
        min_length=5
    )

@tool("start_refund", args_schema=RefundInput)
def start_refund(order_id: str, reason: str) → str:
    """
    Start a refund for an order.

    Only use it after the customer clearly asks for one.
    """
    if order_id.upper() not in ORDERS:
        return f"Cannot refund, no order with id {order_id}."

    return f"Refund started for {order_id.upper()}, reason recorded as: {reason}"
```

The reason for the schema is the same as last week and was repeated in the same words: the **LLM** fills those arguments, so they are not trusted input. `min_length=5` means a refund cannot be started with an empty or one-word reason, and the check happens before the function body runs.

The other three tools

```
@tool
def order_status(order_id: str) → str:
    """Get the item, status and amount of an order using its id."""
    order = ORDERS.get(order_id.upper())
    if order is None:
        return f"No order found with id {order_id}."
    return (f"{order['item']}, status {order['status']}, "
            f"amount {order['amount']} rupees")
```

```
@tool
def delivery_estimate(order_id: str) → str:
    """Estimate when an order will be delivered."""
    order = ORDERS.get(order_id.upper())
    if order is None:
        return f"No order found with id {order_id}."
    return "2 days" if order["pin"].startswith("4") else "5 days"
```

```
@tool
def escalate(order_id: str, note: str) → str:
    """Send the case to a human agent when you cannot solve it."""
    return f"Case for {order_id} sent to a human with note: {note}"
```

AN ESCAPE HATCH IS PART OF THE DESIGN

`escalate` does nothing clever, and that is the point. Give an agent a way to hand a case to a person and it stops being forced to invent an answer when it is out of its depth. Then say so in the system prompt, so it knows when to reach for it.

In this demo it just prints. In a real product it would open a ticket.

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

Everything wired together

```
agent = create_agent(  
    model="openai:gpt-4o-mini",  
    tools=[order_status, delivery_estimate, start_refund, escalate],  
    system_prompt=(  
        "You are the support agent for an online store. "  
        "Always look up an order before answering about it, never guess numbers or dates. "  
        "Start a refund only when the customer asks for one. "  
        "Escalate anything you cannot handle with the tools you have. "  
        "In the final response, clearly state the order id and what action was taken."  
    ),  
    response_format=Reply,  
    checkpoint=InMemorySaver(),  
)  
  
conversation = [  
    "Hi, where is my order ORD-1002?",  
    "When will it reach me?",  
    "That is too late, I want a refund because I need it this week.",  
]  
  
config = {"configurable": {"thread_id": "telusko-new"}}  
  
for question in conversation:  
    result = agent.invoke(  
        {"messages": [{"role": "user", "content": question}]},  
        config=config  
    )
```

```

TURN 1  "where is my order ORD-1002?"
        → order_status tool
        → "Mechanical keyboard, packed, 4999 rupees"

TURN 2  "when will it reach me?"
        no order id in this sentence.
        the thread supplied it.
        → delivery_estimate tool
        → "5 days"

TURN 3  "that is too late, I want a refund because I need it this week"
        no order id here either.
        the thread supplied it again.
        the system prompt allowed it, because the customer asked.
        → start_refund tool
        → "Refund started for ORD-1002"

```

three turns, one thread, one order id mentioned once.

READ THE SYSTEM PROMPT AS POLICY, NOT DECORATION

Four sentences, and each one prevents a real failure. *Always look up* stops invented order details. *Never guess numbers or dates* stops a plausible delivery estimate. *Start a refund only when the customer asks* stops a refund being triggered by a complaint. *Escalate anything you cannot handle* gives it somewhere to go.

A refund tool with no rule around it is a tool that will eventually fire when nobody asked for one.

HONEST STATUS FROM THE LIVE CLASS

This file took a few attempts on screen. The structured output fields were not printing what was expected, and several minutes went into adjusting the `Reply` model and the system prompt before the output looked right. The agent behaviour was correct throughout, it was the printing and the field descriptions that needed work.

There is also a real quirk in the finished file worth knowing about: **only the last turn is printed**. The `result` assignment sits inside the loop but the `print` statements sit outside it, so each turn overwrites the previous one and only the third survives. The trainer explained this when asked. If you want all three turns, move the prints inside the `for` block.

11 Structured output, at the agent level

A paragraph is fine for a person. It is useless to the next function in your program. If a dashboard needs to know the intent and whether a human must follow up, it cannot parse that out of a sentence reliably.

```
from typing import Literal

class Reply(BaseModel):
    """Structured result returned by the support agent."""

    order_id: str = Field(description="Order discussed, or NA")

    intent: Literal["status", "delivery", "refund", "other"] = Field(
        description="What the customer wanted"
    )

    message: str = Field(description="The reply to show the customer")

    action_taken: str = Field(description="What the agent actually did")

    needs_human: bool = Field(description="True if a person must follow up")
```

Pass it as `response_format=Reply` and the filled object comes back under a separate key:

```
reply = result["structured_response"]

print("Customer: ", question)
print("Agent:     ", reply.message)
print()
print("Application data:")
print(f"  order_id      = {reply.order_id}")
print(f"  intent         = {reply.intent}")
print(f"  action_taken   = {reply.action_taken}")
print(f"  needs_human    = {reply.needs_human}")
```

WITHOUT response_format	WITH response_format
<code>result["messages"][-1].content</code>	<code>result["structured_response"]</code>
<pre>"A refund has been initiated for your order ORD-1002..." a string. your code has to parse it. good luck.</pre>	<pre>Reply(order_id = "ORD-1002", intent = "refund", message = "A refund has...", action_taken = "started refund", needs_human = False) a real object. reply.needs_human is a bool. the next if statement just works.</pre>

TWO CLARIFICATIONS FROM THE ROOM

Who produces the structured output? The LLM does, not the agent. The agent arranges it; the model fills it in. That distinction was corrected on air and it is the right one.

Is your Pydantic class sent to the model as Python code? No. LangChain converts the class into a schema in the format that provider's API understands, and sends that. The model never sees your source.

Reference: docs.langchain.com/oss/python/langchain/structured-output

12 Clarifications from the Q&A

- **Who decides which tool to call?** The LLM, every time. It decides based on its training, the question, the system prompt and the tool descriptions. You are not making that decision anywhere in your code.
- **Is there a standard way to write a tool description so the right tool always gets picked?** No guarantee. You can make descriptions much clearer and cut ambiguity a long way, but the final choice stays with the model. Two tools with overlapping purposes will sometimes be confused, however well you write them.
- **What if I skip the docstring on a tool?** The tool is still created and still runs. But the docstring is what the model reads to decide, so leaving it out makes the tool much less likely to be chosen correctly. You can also pass the name and description to `@tool` explicitly instead of relying on the function name and docstring.
- **Does memory increase token cost?** Yes. The whole conversation is sent on every call, so a longer thread costs more, exactly as it does in ChatGPT. This is the same point from session 2, now with a checkpointer doing the resending for you.

- **Is this the same as human in the loop?** No. Nothing here pauses for a person to approve anything. Human in the loop is a separate topic and comes later.
- **Is `create_agent` production ready?** Yes. It was `InMemorySaver` that carries the "not for production" warning, not the agent itself.
- **Do we need MCP before middleware?** No. Middleware comes next. MCP is simply a way to call external tools, and it can come before or after RAG.
- **What is the difference between Pydantic and guardrails?** Pydantic defines and validates a data structure: name must be a string, age must be an integer. Guardrails put rules around what the LLM is allowed to receive and produce, and there are input guardrails and output guardrails. Guardrails can use a schema, but they are a wider idea. Practical examples come later.
- **Given tools like Claude Code exist, is learning to build agents still worth it?** Asked seriously and answered seriously. Using a ready-made agent is not the same as being able to build one you control. You will still need to add your own tools, your own MCP connections and your own retrieval, and harnessing all of that is the skill.
- **Can I get the output as JSON?** Yes, that is what structured output does, and more examples follow.
- **Where do the code files go?** Shared in chat during the session, then added to the course dashboard under the session's resources, with a GitHub link alongside.

13 Key takeaways

- **The loop is the agent.** Send, run whatever tools were requested, append each result, send again, stop when nothing more is asked for.
- **That cycle has a name: ReAct**, from reasoning and acting. Every agent framework is a version of it.
- `tools_by_name` exists because the model returns a string, not a function. You need the lookup.
- **Pass the whole call to `tool.invoke(call)`**, and you get a `ToolMessage` already tied to the request by its id.
- **Read the message trail.** System, Human, AI, Tool, Tool, AI is the whole story of a run, and the first place to look when something is wrong.
- **The number comes from the tool, the sentence comes from the model.** Neither could have produced the answer alone.
- `create_agent` is fifteen lines replaced by three. Input is `{"messages": [ ... ]}`, output is the whole conversation, the answer is `[-1].content`.
- **With several tools the agent chains them**, feeding one result into the next call, and nobody wrote that plan.
- **A good system prompt prevents real failures.** "Never guess, look it up" is why it admitted it did not know a restock date.
- `stream_mode="updates"` is your debugger. One line per step, model and tools alternating.

- **A checkpointer plus a thread id gives you memory.** Same thread, same conversation. Different thread, clean slate. `InMemorySaver` for class, a database saver for production.
- **`create_agent` is built on LangGraph**, which is where the checkpointer comes from and why LangGraph is coming later.
- **`response_format` gives you a validated object**, not a paragraph, so the rest of your program can use it.

COMING NEXT

Agent middleware, described in class as a concept you cannot skip if you understand tool calling. So far the agent is a closed box: model plus tools plus loop. Middleware lets you control what happens around it without rewriting any of it. Change the prompt per request, choose which tools are even offered, retry a tool that failed, keep a long conversation inside budget, route to a different model, protect personal data.

After that: RAG so the agent can search your own documents, human in the loop so it pauses for approval before something risky, then evaluation, tracing and guardrails.

Telusko | Agentic AI Engineering with Python. Notes for Session 05, recorded 5 September 2026.